ASSIGNMENTNO: 3

THREAD SYNCHRONIZATION

AIM : Thread synchronization using counting semaphores and mutual exclusion using mutex.Application to demonstrate: producer-consumer problem with counting semaphores and mutex

OBJECTIVES : To study

- Semaphores
- Mutex
- Producer Consumer Problem

THEORY :

Semaphores:

Semaphore is an integer value used for signaling among processes. Only three operations may be performed on a semaphore, all of which are atomic: initialize, decrement, and increment.

The decrement operation may result in the blocking of a process, and the increment operation may result in the unblocking of a process. It is known as a counting semaphore or a general semaphore. Semaphores are the OS tools for synchronization.

Two types:

1. Binary Semaphore.
2. Counting Semaphore

Binary semaphore

Semaphores which are restricted to the values 0 and 1 (or locked/unlocked, unavailable / available) are called binary semaphores and are used to implement locks.

It is a means of suspending active processes which are later to be reactivated at such time conditions are right for it to continue. A binary semaphore is a pointer which when held by a process grants them exclusive use to their critical section. It is a (sort of) integer variable which can take the values 0 or 1 and be operated upon only by two commands termed in English wait and signal.

Counting semaphore

Semaphores which allow an arbitrary resource count are called counting semaphores.

A counting semaphore comprises: An integer variable, initialized to a value K (K>=0).

During operation it can assume any value <= K, a pointer to a process queue. The queue will hold the PCBs of all those

processes, waiting to enter their critical sections. The queue is implemented as a FCFS, so that the waiting processes are served in a FCFS order.

A counting semaphore can be implemented as follows:

- Initialize – initialize to non negative integer
- Decrement (semWait)
  - Process executes this to receive a signal via semaphore.
  - If signal is not transmitted, process is suspended.
  - Decrements semaphore value
  - If value becomes negative , process is blocked
  - Otherwise it continues execution.
- Increment (semSignal)
  - Process executes it to transmit a signal via semaphore.
  - Increments semaphore value
  - If value is less than or equal to zero, process blocked by semWait is unblocked

```
struct semaphore {
     int count;
     queueType queue;
};
void semWait(semaphore s)
{
     s.count--;
     if (s.count < 0) {
          /* place this process in s.queue */;
          /* block this process */;
     }
}
void semSignal(semaphore s)
{
     s.count++;
     if (s.count <= 0) {
          /* remove a process P from s.queue */;
          /* place process P on ready list */;
     }
}
```

Value of semaphore

- Positive

  Indicates number of processes that can issue wait & immediately continue to execute.

- Zero

  By initialization or because number of processes equal to initial semaphore value have issued a wait Next process to issue a wait is blocked.

- Negative

  Indicates number of processes waiting to be unblocked

  Each signal unblocks one waiting process.

The Producer/Consumer Problem

There are one or more producers generating some type of data (records, characters) and placing these in a buffer. There is a single consumer that is taking items out of the buffer one at a time. The system is to be constrained to prevent the overlap of buffer operations. That is, only one agent (producer or consumer) may access the buffer at any one time. The problem is to make sure that the producer won't try to add data into the buffer if it's full and that the consumer won't try to remove data from an empty buffer. To begin, let us assume that the buffer is infinite and consists of a linear array of elements. In abstract terms, we can define the producer and consumer functions as follows:

```
producer:
while (true) {
   /* produce item v */;
   b[in] = v;
   in++;
}
```

```
consumer:
while (true) {
   while (in <= out)
        /* do nothing */;
   w = b[out];
   out++;
   /* consume item w */;
}
```

Figure illustrates the structure of buffer b. The producer can generate items and store them in the buffer at its own pace. Each time, an index (in) into the buffer is incremented. The consumer proceeds in a similar fashion but must make sure that it does not attempt to read from an empty buffer.

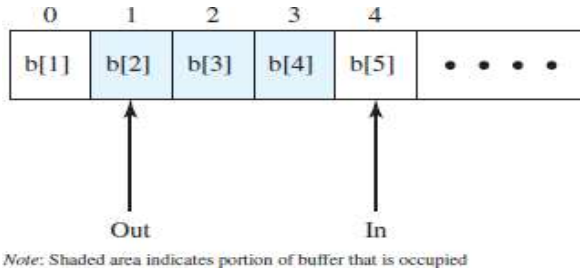


Figure: Infinite buffer for producer/consumer problem

Solution for bounded buffer using counting semaphore

```
/* program boundedbuffer */
const int sizeofbuffer = /* buffer size */;
semaphore s = 1, n= 0, e= sizeofbuffer;
void producer()
{
     while (true) {
          produce();
          semWait(e);
          semWait(s);
          append();
          semSignal(s);
          semSignal(n);
     }
}
void consumer()
{
     while (true) {
          semWait(n);
          semWait(s);
          take();
          semSignal(s);
          semSignal(e);
          consume();
     }
}
void main()
{
     parbegin (producer, consumer);
}
```

POSIX Semaphores

POSIX semaphores allow processes and threads to synchronize their actions. A semaphore is an integer whose value is never allowed to fall below zero. Two operations can be performed on semaphores: increment the semaphore value by one (sem_post(3)); and decrement the semaphore value by one (sem_wait(3)). If the value of a semaphore is currently zero, then a sem_wait(3) operation will block until the value becomes greater than zero.

Semaphore functions:

1. **sem_init()**

It initializes the unnamed semaphore at the address pointed to by sem. The value argument specifies the initial value for the semaphore.

intsem_init(sem_t *sem, intpshared, unsigned int value);

2. sem_wait()

It decrements (locks) the semaphore pointed to by sem. If the semaphore's value is greater than zero, then the decrement proceeds, and the function returns, immediately. If the semaphore currently has the value zero, then the call blocks until it becomes possible to perform the decrement.

intsem_wait(sem_t *sem);

3. sem_post()

It increments (unlocks) the semaphore pointed to by sem. If the semaphore's value consequently becomes greater than zero, then another process or thread blocked in a sem_wait(3) call will be woken up and proceed to lock the semaphore.

intsem_post(sem_t *sem);

1. sem_unlink

It removes the named semaphore referred to by name. The semaphore name is removed immediately. The semaphore is destroyed once all other processes that have the semaphore open close it.

intsem_unlink(const char *name)

All the above functions returns

0 : Success

-1 : Error

Mutex:

Mutexes are a method used to be sure two threads, including the parent thread, do not attempt to access shared resource at the same time. A mutex lock allows only one thread to enter the part that's locked and the lock is not shared with any other processes.

1. pthread_mutex_init()

The function shall initialize the mutex referenced by mutex with attributes specified by attr. If attr is NULL, the default mutex attributes are used; the effect shall be the same as passing the address of a default mutex attributes object. Upon successful initialization, the state of the mutex becomes initialized and unlocked.

intpthread_mutex_init(pthread_mutex_t *restrict mutex, cons pthread_mutexattr_t*restrict attr);

2. **pthread_mutex_lock()**

   The mutex object referenced by mutex shall be locked by calling pthread_mutex_lock(). If the mutex is already locked, the calling thread shall block until the mutex becomes available. This operation shall return with the mutex object referenced by mutex in the locked state with the calling thread as its owner.

   intpthread_mutex_lock(pthread_mutex_t * mutex);

3. **pthread_mutex_unlock()**

   The function shall release the mutex object referenced by mutex. The manner in which a mutex is released is dependent upon the mutex's type attribute. If there are threads blocked on the mutex object referenced by mutex when pthread_mutex_unlock() is called, resulting in the mutex becoming available, the scheduling policy shall determine which thread shall acquire the mutex.

   intpthread_mutex_unlock(pthread_mutex_t * mutex);

4. **pthread_mutex_destroy()**

   The function shall destroy the mutex object referenced by mutex; the mutex object becomes, in effect, uninitialized. A destroyed mutex object can be reinitialized using pthread_mutex_init(); the results of otherwise referencing the object after it has been destroyed are undefined.

   intpthread_mutex_destroy(pthread_mutex_t *mutex);

Program 3:

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

#define BUFFER_SIZE 5

int buffer[BUFFER_SIZE];
int in = 0, out = 0;

sem_t empty;   // Counting semaphore for empty spaces
sem_t full;    // Counting semaphore for filled spaces
pthread_mutex_t mutex;   // Mutex for mutual exclusion

// Producer function
void *producer(void *arg)
{
   int item;
```

```
    for (item = 1; item <= 10; item++)
    {
        // Wait for an empty space
        sem_wait(&empty);

        // Lock the buffer
        pthread_mutex_lock(&mutex);

        buffer[in] = item;
        printf("Producer produced: %d\n", item);
        in = (in + 1) % BUFFER_SIZE;

        // Unlock the buffer
        pthread_mutex_unlock(&mutex);

        // Signal that an item is available
        sem_post(&full);

        sleep(1);
    }

    return NULL;
}

// Consumer function
void *consumer(void *arg)
{
    int item;

    for (int i = 1; i <= 10; i++)
    {
        // Wait for an available item
        sem_wait(&full);

        // Lock the buffer
        pthread_mutex_lock(&mutex);

        item = buffer[out];
        printf("Consumer consumed: %d\n", item);
        out = (out + 1) % BUFFER_SIZE;

        // Unlock the buffer
        pthread_mutex_unlock(&mutex);

        // Signal that an empty space is available
        sem_post(&empty);

        sleep(2);
    }

    return NULL;
}

int main()
```

```
{
    pthread_t producer_thread, consumer_thread;

    // Initialize semaphores
    sem_init(&empty, 0, BUFFER_SIZE);
    sem_init(&full, 0, 0);

    // Initialize mutex
    pthread_mutex_init(&mutex, NULL);

    // Create threads
    pthread_create(&producer_thread, NULL, producer, NULL);
    pthread_create(&consumer_thread, NULL, consumer, NULL);

    // Wait for threads to complete
    pthread_join(producer_thread, NULL);
    pthread_join(consumer_thread, NULL);

    // Destroy semaphores and mutex
    sem_destroy(&empty);
    sem_destroy(&full);
    pthread_mutex_destroy(&mutex);

    return 0;
}
```

CONCLUSION:

Thus, we have implemented producer-consumer problem using ‘C’ in Linux.